UNIVERSIDADE PARANAENSE - UNIPAR

Reconhecida pela Portaria - MEC nº 1580, de 09/11/93 - D.O.U. 16/11/93
Mantenedora: Associação Paranaense de Ensino e Cultura – APEC
CURSO DE ADS

MATÉRIA

DEVOPS TOOLS

ACADÊMICO: MARLLON GUSTAVO DA SILVA
RA: 60001928
ACADÊMICO: GUSTAVO EMANUEL SCANDOLARA MORGENSTER
RA: 60000694

Caso EntregaJá

PROFESSOR: PATRICK JOSÉ CARDOSO

TOLEDO - PR

2026

UNIVERSIDADE PARANAENSE - UNIPAR
Reconhecida pela Portaria - MEC nº 1580, de 09/11/93 - D.O.U. 16/11/93
Mantenedora: Associação Paranaense de Ensino e Cultura – APEC
CURSO DE ADS

Análise de Cenário: Startup EntregaJá

1. Diagnóstico: quais são os problemas?

O jeito que a equipe trabalha pode ter servido no começo, mas já está causando perda de trabalho e problemas para os clientes. No caso apresentado, três pontos precisam de atenção:

- **Código compartilhado por pendrive e e-mail:** cada desenvolvedor acaba com uma cópia diferente do projeto, dificultando saber o que está atualizado. Isso favorece situações como a de Bruno sobrescrever o trabalho de Carla. Esse problema prejudica o **Primeiro Caminho (Fluxo)**, pois gera retrabalho e atrasa as entregas.
- **Falta de histórico e identificação das versões:** ninguém sabe exatamente quais alterações estão em produção ou quando o problema começou. Isso dificulta investigar as reclamações dos clientes e está relacionado ao **Segundo Caminho (Feedback)**, já que a equipe demora para entender e corrigir as falhas.
- **Publicação manual concentrada em Ana:** depender de uma única pessoa copiando arquivos por FTP cria um gargalo. Além da sobrecarga, existe o risco de esquecer arquivos ou enviar uma versão incompleta. Esse ponto está ligado ao pilar de **Automação do CALMS**, pois uma tarefa repetitiva e sujeita a erros ainda é feita manualmente.

2. Proposta: como a equipe poderia trabalhar?

Com Git e GitHub, a equipe pode adotar um fluxo simples, em seis passos:

1. **Clonar o repositório:** Ana, Bruno e Carla fazem uma cópia do repositório oficial do GitHub em seus computadores.
2. **Criar uma branch:** antes de começar uma tarefa, cada pessoa atualiza sua cópia e cria uma branch para trabalhar na alteração sem mexer diretamente na versão principal.

3. **Registrar as alterações:** durante o desenvolvimento, cada pessoa faz commits com mensagens claras, explicando o que mudou.
4. **Enviar para o GitHub:** com o `git push`, as alterações ficam disponíveis para o restante da equipe.
5. **Revisar e integrar:** o desenvolvedor abre um Pull Request. Outro integrante revisa o código, a equipe verifica os testes e resolve possíveis conflitos antes de juntar a alteração à branch principal.
6. **Atualizar a produção:** a equipe identifica a versão aprovada com uma tag e publica essa versão seguindo um procedimento definido. Além de atualizar o código, é necessário conferir as dependências, reiniciar o serviço quando necessário e verificar se a aplicação está funcionando. A ideia é automatizar essa etapa com CI/CD.

3. Recuperação: como trazer a tela de Carla de volta?

Se Carla tivesse salvo sua tela de cupons em um commit, o trabalho estaria registrado no histórico do Git.

A equipe poderia usar `git log` para localizar os commits e verificar qual alteração removeu ou sobrescreveu a tela. Se fosse adequado desfazer todo o commit de Bruno, bastaria usar `git revert`, informando o identificador desse commit. Isso criaria um novo registro desfazendo a alteração problemática, sem apagar o histórico.

Se o mesmo commit também tivesse uma correção de frete que precisasse ser mantida, a equipe poderia recuperar apenas os arquivos ou trechos da tela de cupons e fazer um novo commit.

Assim, a recuperação poderia acontecer em poucos minutos, seguida de testes e da publicação da versão corrigida. O ponto principal é que **o trabalho de Carla precisaria ter sido registrado no Git antes de ser sobrescrito**.

4. Risco: o que o Git não resolve sozinho?

O Git não resolve, por conta própria, o risco da publicação manual por FTP. Mesmo com o código organizado, Ana ainda pode esquecer um arquivo, copiar algo para a pasta errada ou ter a transferência interrompida.

Para resolver esse problema, a equipe precisa automatizar a entrega com um pipeline de CI/CD, que faça as verificações e publique a versão aprovada de forma padronizada. O Git organiza as alterações; a automação ajuda a colocá-las em produção com mais segurança.

